

Designing Machine Learning Systems

by Chip Huyen — Complete Deep Notes with MLOps, Cloud Architecture, Data Paradigms & Memory Boosters (v3)

1. 🌐 Big Picture First

Core Message

The model is only **5% of a production ML system**. The remaining 95% is data pipelines, feature stores, serving infrastructure, monitoring, and continual learning loops. Chip Huyen's central thesis: *building ML systems is a software engineering discipline, not just a modelling exercise.*

Author's Purpose

To give practitioners a holistic, iterative framework for designing ML systems that are **reliable, scalable, maintainable, and adaptable** — from raw data ingestion all the way to production monitoring and retraining.

9-Step End-to-End Flow

#	Step	Key Question
1	Problem Framing	Is ML the right tool? What is the business objective?
2	Data Engineering	Where does data come from? How is it stored and moved?
3	Training Data	How do we label, sample, and handle imbalance?
4	Feature Engineering	What signals matter? How do we avoid leakage?
5	Model Development	Which model family? How do we tune and track experiments?
6	Offline Evaluation	Does the model generalise? Is it calibrated?
7	Deployment & Serving	Batch or online? Edge or cloud? Compressed or full?
8	Monitoring & Observability	Is the model still working in production?
9	Continual Learning	How and when do we retrain?

2. Story-Based Explanation

The Netflix Story — A Journey Through All 9 Steps

Step 1 – Problem Framing: Netflix wants to reduce churn. The business metric is *monthly retention rate*. The ML metric is *click-through rate (CTR)* on recommended titles. The team frames it as a ranking problem.

Step 2 – Data Engineering: User events (clicks, watch-time, ratings) stream in via Kafka. Structured metadata (title, genre, release year) lives in PostgreSQL. Video thumbnails and trailers sit in S3. An ETL pipeline lands everything in a data lake (S3 + Delta Lake).

Step 3 – Training Data: Implicit labels: a watch > 70% = positive signal. The team uses stratified sampling to handle the long tail of niche content. They track data lineage with DVC so every training snapshot is reproducible.

Step 4 – Feature Engineering: User embeddings (watch history), item embeddings (content metadata), and contextual features (time of day, device). Features are computed in Spark, stored in Feast (feature store), and served with <10 ms latency at inference time.

Step 5 – Model Development: A Two-Tower neural network retrieves candidates; a LightGBM ranker re-ranks them. Experiments are tracked in MLflow. DVC versions datasets and model artefacts.

Step 6 – Offline Evaluation: NDCG@10 on a held-out time-based split. Slice-based evaluation ensures the model is fair across age groups and geographies.

Step 7 – Deployment: The model is containerised in Docker, orchestrated on Kubernetes (EKS), and served via TorchServe behind an API Gateway. A canary deployment rolls out to 5% of traffic first.

Step 8 – Monitoring: Prometheus tracks latency and error rates. Evidently AI monitors feature drift and prediction drift. Alerts fire in PagerDuty when PSI > 0.2.

Step 9 – Continual Learning: A weekly stateful retraining pipeline is triggered by Airflow. If drift is detected mid-week, an emergency retrain is triggered automatically.

3. Structured Smart Notes

3.1 Problem Framing

- Always map **business objective** → **ML objective** → **ML metric**
- Use ML only when: rules are too complex, patterns change over time, scale is large
- Avoid ML when: problem is solvable with simple heuristics, data is scarce, interpretability is legally required

- Decoupled objectives: use separate models for separate goals, then combine with a weighted formula

3.2 Data Engineering Fundamentals

Concept	Key Points
Data Sources	User-generated, system logs, third-party, internal databases
Data Formats	Row (CSV, JSON) vs Column (Parquet, ORC) — columnar is faster for analytics
Data Models	Relational (SQL), Document (MongoDB), Graph (Neo4j), Wide-Column (Cassandra)
ETL vs ELT	ETL: transform before load (traditional); ELT: load raw, transform later (modern cloud)
Dataflow Modes	Databases, Services (REST/gRPC), Real-time transport (Kafka, Kinesis)

3.3 Training Data

Topic	Key Points
Sampling	Non-probability (convenience, snowball) vs Probability (random, stratified, weighted, reservoir, importance)
Labeling	Hand labeling (expensive, slow) → Natural labels (implicit feedback) → Programmatic (Snorkel)
Class Imbalance	Resampling (over/under), class weights, SMOTE, threshold moving, anomaly detection framing
Data Augmentation	NLP: back-translation, synonym swap; CV: flip, crop, colour jitter, mixup, cutmix
Weak Supervision	Labelling functions + label model (Snorkel) — fast, noisy but scalable

3.4 Feature Engineering

- **Missing values:** MCAR (safe to drop), MAR (impute), MNAR (create indicator column)
- **Scaling:** Min-max normalisation, Z-score standardisation, log transform for skewed data
- **Encoding:** OHE (low cardinality), Target encoding (high cardinality), Embeddings (very high cardinality)
- **Feature crossing:** Combine features to capture interactions (e.g., city × time_of_day)
- **Data leakage:** Future data leaking into training — always split by time, not randomly
- **Feature selection:** Filter (correlation), Wrapper (RFE), Embedded (LASSO, tree importance), SHAP

3.5 Model Development & Offline Evaluation

Task	Recommended Models	Primary Metric
Binary Classification	Logistic Regression, XGBoost, Neural Net	AUC-ROC, F1
Multi-class	Softmax NN, Random Forest	Macro F1, Top-K Accuracy
Regression	Linear, Gradient Boosting, NN	RMSE, MAE, MAPE
Ranking	LambdaMART, Two-Tower NN	NDCG, MRR, MAP
NLP	BERT, GPT, T5	BLEU, ROUGE, Perplexity
Computer Vision	ResNet, EfficientNet, ViT	mAP, IoU, FID

3.6 Deployment & Serving

Mode	Latency	Use Case
Batch Prediction	Hours	Email campaigns, offline scoring
Online Prediction	Milliseconds	Real-time recommendations, fraud detection
Streaming Prediction	Seconds	IoT, live dashboards
Edge Prediction	Sub-ms	Mobile apps, autonomous vehicles

3.7 Monitoring & Distribution Shifts

- **Covariate shift:** $P(X)$ changes, $P(Y|X)$ stays same → retrain with new data
- **Label shift:** $P(Y)$ changes → adjust class priors
- **Concept drift:** $P(Y|X)$ changes → full model refresh needed
- **Detection:** PSI, KS test, MMD, chi-squared test, CUSUM
- **Monitoring stack:** Prometheus + Grafana (ops), Evidently AI / WhyLogs (ML drift)

4. Data Paradigms — Structured vs Unstructured

4.1 Structured vs Unstructured Data

Dimension	Structured	Unstructured
Definition	Predefined schema, tabular rows/columns	No fixed schema — text, images, audio, video
Storage	RDBMS (PostgreSQL, MySQL), Data Warehouse	Object store (S3, GCS), NoSQL, Data Lake

Query Language	SQL	Custom APIs, vector search, full-text search
ML Use	Tabular models, time-series	Deep learning (NLP, CV, speech)
Volume	GBs–TBs	TBs–PBs
Examples	Transactions, user profiles, sensor readings	Emails, images, logs, social media posts

4.2 SQL vs NoSQL

Dimension	SQL (Relational)	NoSQL
Schema	Fixed, enforced	Flexible, dynamic
ACID	Full ACID compliance	Eventual consistency (BASE)
Scaling	Vertical (scale up)	Horizontal (scale out)
Joins	Native, efficient	Avoided by design (denormalised)
Best For	Transactions, reporting, structured analytics	High-throughput writes, flexible schemas, large scale
Examples	PostgreSQL, MySQL, BigQuery	MongoDB, Cassandra, Redis, DynamoDB, Pinecone

4.3 NoSQL Types for ML

Type	Example	ML Use Case
Document	MongoDB	User profiles, product catalogues
Key-Value	Redis, DynamoDB	Feature store online layer, session cache
Wide-Column	Cassandra, HBase	Time-series events, clickstream
Graph	Neo4j	Fraud detection, knowledge graphs
Vector	Pinecone, Weaviate, Milvus	Embedding search, RAG, semantic similarity
Time-Series	InfluxDB, TimescaleDB	IoT, monitoring metrics, financial data

4.4 Batch vs Real-Time Processing

Dimension	Batch (Spark, Hive)	Real-Time (Flink, Kafka Streams)
Latency	Minutes to hours	Milliseconds to seconds
Throughput	Very high	Moderate to high
Complexity	Lower	Higher

Cost	Lower	Higher
Use Case	Offline training, historical analytics	Fraud detection, live recommendations
Tools	Spark, Hive, dbt, Airflow	Kafka, Flink, Spark Streaming, Kinesis

5. 10-Layer Cloud-Based ML Architecture

Layer 1 — Data Sources

- **Structured:** Transactional DBs (PostgreSQL, MySQL), ERP/CRM systems, IoT sensors
- **Unstructured:** User uploads (S3), social media APIs, web scraping, video/audio streams
- **Semi-structured:** JSON logs, XML feeds, Parquet files

Layer 2 — Data Ingestion

- **Batch ingestion:** AWS Glue, Apache Sqoop, Fivetran, Airbyte
- **Stream ingestion:** Apache Kafka, AWS Kinesis, Google Pub/Sub, Azure Event Hubs
- **CDC (Change Data Capture):** Debezium for real-time DB replication

Layer 3 — Storage

- **Data Lake:** AWS S3 + Delta Lake / Apache Iceberg (raw + processed zones)
- **Data Warehouse:** Snowflake, BigQuery, Redshift (structured analytics)
- **Lakehouse:** Databricks (combines lake + warehouse)
- **Object Store:** S3, GCS, Azure Blob (images, video, model artefacts)
- **OLTP DB:** PostgreSQL / MySQL (transactional structured data)
- **NoSQL:** MongoDB (documents), Cassandra (events), Redis (cache), Pinecone (vectors)

Layer 4 — Data Processing

- **Batch:** Apache Spark (PySpark), dbt (SQL transforms), AWS EMR
- **Stream:** Apache Flink, Spark Structured Streaming, Kafka Streams
- **Orchestration:** Apache Airflow, Prefect, Dagster, AWS Step Functions
- **Data Quality:** Great Expectations, Deequ, Monte Carlo

Layer 5 — Feature Store

- **Offline store:** Parquet on S3 / BigQuery — used for training
- **Online store:** Redis / DynamoDB — used for low-latency inference
- **Tools:** Feast, Tecton, Hopsworks, AWS SageMaker Feature Store
- **Key benefits:** Feature reuse, point-in-time correctness, no training-serving skew

Layer 6 — Model Training, Validation & Versioning

- **Training:** AWS SageMaker, Vertex AI, Azure ML, Databricks MLflow
- **Distributed training:** Horovod, PyTorch DDP, DeepSpeed (for large models)
- **Experiment tracking:** MLflow, Weights & Biases, Neptune

- **Data versioning:** DVC (Data Version Control) — tracks datasets + model artefacts in Git
- **Model registry:** MLflow Model Registry, SageMaker Model Registry
- **Validation:** Offline metrics, slice-based evaluation, bias audits, shadow testing

Layer 7 — Containerisation & Orchestration

- **Docker:** Package model + dependencies into portable containers
- **Kubernetes (K8s):** Orchestrate containers at scale; auto-scaling, self-healing, rolling updates
- **Helm:** K8s package manager for ML deployments
- **Kubeflow:** ML-specific K8s pipelines (training, serving, hyperparameter tuning)
- **Cloud K8s:** AWS EKS, Google GKE, Azure AKS

Layer 8 — Model Serving & Inference

- **REST/gRPC APIs:** FastAPI, Flask, TorchServe, TF Serving, Triton Inference Server
- **Serverless:** AWS Lambda, Google Cloud Functions (low-traffic models)
- **Batch scoring:** Spark batch jobs, SageMaker Batch Transform
- **Edge:** TensorFlow Lite, ONNX Runtime, CoreML (mobile/IoT)
- **Model compression:** Quantisation (INT8), Pruning, Knowledge Distillation, ONNX export

Layer 9 — CI/CD for ML (MLOps)

- **CI:** GitHub Actions / Jenkins — lint, unit tests, data validation on every commit
- **CD:** Automated model deployment after passing evaluation gates
- **CT (Continuous Training):** Triggered by schedule, drift detection, or data volume thresholds
- **Deployment strategies:** Shadow → Canary → Blue-Green → A/B Test → Multi-Armed Bandit

Layer 10 — Monitoring, Observability & Continual Learning

- **Operational metrics:** Latency (p50/p95/p99), throughput, error rate, CPU/GPU utilisation
- **ML metrics:** Prediction drift, feature drift, data quality, model accuracy
- **Tools:** Prometheus + Grafana, Evidently AI, WhyLogs, Arize, Fiddler
- **Alerting:** PagerDuty, OpsGenie, Slack webhooks
- **Continual learning:** Stateless (full retrain) vs Stateful (fine-tune from last checkpoint)

End-to-End Flow Diagram

```
[Data Sources] → [Ingestion: Kafka/Glue] → [Storage: S3/Snowflake/Redis]
  → [Processing: Spark/Flink] → [Feature Store: Feast/Tecton]
  → [Training: MLflow/SageMaker + DVC] → [Docker + Kubernetes]
  → [Serving: FastAPI/TorchServe/Triton]
  → [CI/CD: GitHub Actions] → [Deployment: Shadow→Canary→A/B]
```


→ [Monitoring: Prometheus+Evidently] → [Continual Learning Loop]
↑_____|

Cloud Provider Comparison

Component	AWS	GCP	Azure
ML Platform	SageMaker	Vertex AI	Azure ML
Stream Ingestion	Kinesis	Pub/Sub	Event Hubs
Data Warehouse	Redshift	BigQuery	Synapse Analytics
Object Storage	S3	GCS	Blob Storage
Kubernetes	EKS	GKE	AKS
Serverless	Lambda	Cloud Functions	Azure Functions
Feature Store	SageMaker FS	Vertex Feature Store	Azure ML FS
Monitoring	CloudWatch	Cloud Monitoring	Azure Monitor
Workflow Orchestration	Step Functions	Cloud Composer	Azure Data Factory
Container Registry	ECR	Artifact Registry	ACR

6. MLOps — Complete Deep Dive

6.1 What is MLOps?

MLOps (Machine Learning Operations) is the discipline of applying DevOps principles to ML systems — automating the entire lifecycle from data ingestion to model deployment, monitoring, and retraining. It bridges the gap between ML experimentation and reliable production systems.

Core MLOps Pillars:

-  **Reproducibility** — same code + same data = same model, always
-  **Automation** — eliminate manual steps in training, evaluation, and deployment
-  **Monitoring** — detect model degradation before users notice
-  **Continuous Improvement** — retrain automatically when performance drops
-  **Collaboration** — data scientists, ML engineers, and DevOps work from the same platform

6.2 MLOps Maturity Model (Level 0 → Level 3)

Level	Name	Characteristics	Who Does This
Level 0	Manual Process	All steps manual; scripts in notebooks; no versioning; model deployed once and forgotten; no monitoring	Early-stage startups, academic projects
Level 1	ML Pipeline Automation	Automated training pipeline; experiment tracking; model registry; basic monitoring; manual deployment trigger	Growing ML teams, mid-size companies
Level 2	CI/CD Pipeline Automation	Automated CI/CD for ML; automated testing (data + model); automated deployment; drift-triggered retraining; full observability	Mature ML teams, large enterprises
Level 3	Full MLOps Automation	Self-healing systems; automated A/B testing; bandit-based deployment; real-time feature engineering; automated governance and compliance	Big Tech (Google, Netflix, Uber, Airbnb)

Key Insight: Most companies operate at Level 0 or Level 1. Moving to Level 2 provides the biggest ROI — it reduces model deployment time from weeks to hours and catches production failures automatically.

6.3 CI/CD/CT for ML — Detailed Breakdown

Stage	What It Does	Tools	Triggers
-------	--------------	-------	----------

CI (Continuous Integration)	Code linting, unit tests, data validation, feature schema checks, model training smoke test	GitHub Actions, Jenkins, CircleCI, GitLab CI	Every code commit / PR
CD (Continuous Delivery)	Package model into Docker, push to registry, deploy to staging, run integration tests, promote to production	ArgoCD, Spinnaker, AWS CodeDeploy, Helm	After CI passes + manual approval gate
CT (Continuous Training)	Retrain model on new data, evaluate against champion model, auto-promote if challenger wins	Airflow, Kubeflow Pipelines, Vertex AI Pipelines, SageMaker Pipelines	Schedule (daily/weekly), drift alert, data volume threshold

6.4 MLOps Tooling Landscape — Full Comparison

Category	Tool	Strengths	Weaknesses
Experiment Tracking	MLflow	Open-source, language-agnostic, model registry built-in	UI is basic; scaling requires setup
Experiment Tracking	Weights & Biases (W&B)	Best-in-class UI, collaboration features, sweeps for HPO	Paid for large teams; data leaves your infra
Experiment Tracking	Neptune	Great for large teams, metadata management	Less popular, smaller community
Pipeline Orchestration	Apache Airflow	Mature, huge ecosystem, Python-native DAGs	Not ML-specific; complex setup; no dynamic pipelines
Pipeline Orchestration	Prefect	Modern Python API, dynamic workflows, cloud-native	Smaller ecosystem than Airflow
Pipeline Orchestration	Dagster	Asset-based paradigm, great observability, type-safe	Steeper learning curve
ML Pipelines	Kubeflow Pipelines	K8s-native, scalable, supports distributed training	Complex setup; requires K8s expertise
ML Pipelines	ZenML	Framework-agnostic, stack-based, easy to switch backends	Newer, smaller community
ML Pipelines	Metaflow (Netflix)	Python-first, great for data scientists, AWS-native	AWS-centric; less flexible on other clouds
ML Platform (Managed)	AWS SageMaker	End-to-end managed, deep AWS integration	Vendor lock-in; expensive at scale
ML Platform (Managed)	Google Vertex AI	Best AutoML, tight BigQuery integration	GCP-only; complex pricing
	Azure ML		

ML Platform (Managed)		Enterprise-grade, great for Microsoft shop
--------------------------	--	---